

```
print('Each person eats', slices / eaters, 'slices.')
ZeroDivisionError: division by zero
```

It's helpful to remember that the line number the traceback mentions is only the point at which the Python interpreter detected an error. The true cause of the error might be on the previous line of code or even much earlier in the program.

Syntax errors in the source code are caught by the interpreter before the program runs, but syntax errors can also happen at runtime. The `eval()` function can take a string of Python code and run it, which might produce a `SyntaxError` at runtime. For example, `eval('print("Hello, world")')` is missing a closing double quote, which the program won't encounter until the code calls `eval()`.

A *semantic error* (also called a *logical error*) is a more subtle bug. Semantic errors won't cause error messages or crashes, but the computer carries out instructions in a way the programmer didn't intend. In English, the equivalent of a semantic error would be telling the computer, "Buy a carton of milk from the store and if they have eggs, buy a dozen." The computer would then buy 13 cartons of milk because the store had eggs. For better or worse, computers do exactly what you tell them to. For example, enter the following into the interactive shell:

```
>>> print('The sum of 4 and 2 is', '4' + '2')
```

You would get the following output:

```
The sum of 4 and 2 is 42
```

Obviously, 42 isn't the answer. But notice that the program didn't crash. Because Python's `+` operator adds integer values and concatenates string values, mistakenly using the string values `'4'` and `'2'` instead of integers caused unintended behavior.

Parameters vs. Arguments

Parameters are the variable names between the parentheses in a `def` statement. *Arguments* are the values passed in a function call, which are